Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторные работы по курсу «Информационный поиск»

Студент: З. Д. Брюханов
Преподаватель: А. А. Кухтичев
Группа: М8О-407Б-22
Дата: 05.03.2026
Оценка: _____
Подпись: _____

Москва, 2026

1 Добыча корпуса документов

1.1 Цель работы

Подготовить корпус документов для дальнейшего использования в задачах информационного поиска: выбрать источники, скачать данные, выделить текстовое содержимое и получить статистику.

1.2 Источники данных

Для формирования корпуса были выбраны два источника, объединённые тематикой «мобильная разработка»:

1. **Habr.com** — IT-платформа, хабы «Мобильная разработка», «Разработка под Android», «Разработка под iOS». Статьи содержат технический текст на русском языке с вкраплениями английских терминов и фрагментов кода.

2. **Wikipedia (ru)** — русскоязычная Википедия, категории «Мобильное программное обеспечение», «Программное обеспечение для Android», «Программное обеспечение для iOS». Энциклопедический стиль, структурированный текст, густая сеть внутренних ссылок.

Выбор именно двух разнородных источников позволяет охватить и профессиональный (блоговый), и энциклопедический стиль изложения, что делает корпус более представительным.

1.3 Характеристика документов

Формат данных — HTML5 в кодировке UTF-8. Основной контент размещён в тегах `<p>`, `<h1>`–`<h6>`, списках `<ul>`/`<ol>`. На Habr часто встречаются блоки кода (`<pre><code>`), на Wikipedia — внутренние ссылки, сноски и таблицы.

Мета-информация присутствует в тегах `<meta>` (описание, ключевые слова), а также в разметке Open Graph (`og:title`, `og:description`).

1.4 Выделение текста

Очистка HTML выполняется библиотекой BeautifulSoup4 (Python): удаляются теги `<script>`, `<style>`, `<nav>`, `<footer>`, `<header>`, `<aside>`, `<noscript>`, после чего извлекается текстовое содержимое.

1.5 Существующие поисковики

Для проверки доступности документов использовались:

- Встроенный поиск Habr (`habr.com/ru/search`).
- Поиск Google с оператором `site:`: `site:habr.com "flutter разработка"`.
- Встроенный поиск Википедии.

Пример запроса: `site:habr.com "android kotlin coroutines"`.

Результат: Google находит релевантные статьи, однако в выдачу попадают страницы профилей пользователей, комментарии и дубликаты. Кроме того, невозможно выполнить булев запрос с оператором NOT по ограниченному подмножеству документов.

Пример запроса: `site:ru.wikipedia.org "мобильное приложение"`.

Результат: Найдена основная статья и связанные страницы, но в выдачу попадают служебные страницы («Обсуждение:», «Шаблон:»).

1.6 Статистика корпуса

Параметр	**Значение**
Количество документов	30 000
Общий размер сырых данных (HTML)	6.84 ГБ (7 340 032 000 байт)
Общий размер чистого текста	2.18 ГБ (2 340 716 544 байт)
Средний размер сырого документа	238.9 КБ
Средний объём текста в документе	76.2 КБ
Доля полезного текста	31.9%

Таблица 1: Характеристики корпуса документов

1.7 Вывод

Выбранные источники удовлетворяют требованиям: тематическая однородность (мобильная разработка), достаточный объём (30 000 документов), наличие внешних поисковиков для проверки. Доля полезного текста составляет около 32%, остальное — HTML-разметка, скрипты и стили, которые фильтруются при предобработке.

2 Поисковый робот

2.1 Цель работы

Разработать краулер, который обходит веб-страницы по ссылкам, скачивает HTML и сохраняет в базу данных. Робот должен поддерживать возобновление работы после остановки.

2.2 Метод решения

Краулер реализован на Python. В качестве хранилища выбрана СУБД SQLite — весь корпус хранится в одном файле `crawler_data.db`, что упрощает перенос и дальнейшую обработку на C++.

Конфигурация задаётся в файле `config.yaml`:

```
db:
  path: "crawler_data.db"
logic:
  request_delay: 0.15
  max_docs: 30000
  seed_urls:
    - "https://habr.com/ru/hubs/mobile_dev/articles/"
    - "https://habr.com/ru/hubs/android_dev/articles/"
    - "https://habr.com/ru/hubs/ios_dev/articles/"
    - "https://ru.wikipedia.org/wiki/..."
  allowed_domains:
    - "habr.com"
    - "ru.wikipedia.org"
```

2.3 Архитектура

Робот работает в режиме BFS (обход в ширину). Состояние хранится в двух таблицах SQLite:

- `pages` — скачанные документы (id, url, source_name, html_content, fetched_at).
- `crawl_queue` — очередь URL со статусами: `waiting` → `active` → `done`.

При запуске робот проверяет состояние очереди и продолжает с того URL, на котором остановился. При повторном запуске уже скачанные документы не перекачиваются.

2.4 Фильтрация URL

Отсекаются ссылки на изображения (`.jpg`, `.png`, `.svg`), PDF, архивы, а также служебные страницы Wikipedia (`Special:`, `Служебная:`, `Обсуждение:`, `Файл:`, `Шаблон:`) и ссылки с параметрами редактирования (`action=edit`).

2.5 Журнал выполнения

В процессе реализации возникли проблемы:

1. **Ловушки ссылок** — бесконечные ссылки на календари и служебные страницы Wikipedia. Решено расширением списка фильтров URL.

2. **Блокировка запросов (HTTP 429/403)**. Решено добавлением задержки между запросами (`request_delay: 0.15`) и корректного User-Agent.

3. **Таймауты**. Некоторые страницы отвечают медленно. Установлен таймаут 10 секунд на запрос, при ошибке URL пропускается.

2.6 Тестирование

№	Сценарий	Ожидание	Результат
1	Первый запуск	БД создана, данные начали поступать	Успешно
2	Фильтрация	Ссылки на .jpg, Special: не попадают в очередь	Успешно
3	Возобновление	После Ctrl+C и повторного запуска робот продолжает	Успешно
4	Ошибка 404	Страница пропускается, робот продолжает	Успешно

Таблица 2: Результаты тестирования краулера

2.7 Вывод

Краулер успешно собирает корпус из двух источников с возможностью остановки и возобновления. SQLite в качестве хранилища обеспечивает надёжность (ACID) и удобство доступа из C++ на этапе индексации. Задержка между запросами и фильтрация URL предотвращают блокировки и ловушки.

3 Токенизация

3.1 Цель работы

Реализовать алгоритм разбиения текста HTML-документов на токены (слова) и получить статистические характеристики процесса.

3.2 Правила токенизации

Токенизация реализована на C++ в файле `tokenizer.cpp`. Алгоритм выполняет однопроходную обработку:

1. **Удаление HTML-тегов**: всё между `<` и `>` заменяется на пробел, чтобы не склеивались соседние слова.
2. **Набор разделителей**: пробельные символы (ASCII ≤ 32), знаки препинания (`. , ! ? : ; ‘‘ ( ) [ ] { } < > / \ | - + = # @ * ~ ‘ ^`).
3. **Нормализация**: латинские символы приводятся к нижнему регистру (A–Z $\rightarrow$ a–z). Кириллица передаётся как есть (UTF-8, без преобразования регистра в текущей реализации).
4. **Формирование токена**: любая непрерывная последовательность символов, не являющихся разделителями.

3.3 Достоинства и недостатки

Достоинства:

- Линейная сложность $O(N)$ — один проход по тексту без возвратов.
- Высокая скорость за счёт отсутствия регулярных выражений.
- Нормализация регистра уменьшает размер словаря.

Недостатки:

- Технические термины (`C++`, `C#`) распадаются на отдельные символы, так как `+` и `#` считаются разделителями.
- URL-адреса, оставшиеся в тексте, порождают мусорные токены.
- Кириллический регистр не нормализуется — «Андроид» и «андроид» считаются разными токенами.

Возможные улучшения: добавить список исключений для технических терминов, ввести фильтрацию по минимальной длине токена и нормализацию кириллического регистра через ICU.

3.4 Результаты

Зависимость времени от объёма данных линейная. Узким местом является чтение из SQLite, а не сама токенизация.

Параметр	**Значение**
Документов обработано	30 000
Размер чистого текста	2 233 080 КБ (≈2.18 ГБ)
Общее число токенов	645 759 412
Средняя длина токена	9.05 символов
Время обработки	241.3 сек
Скорость токенизации	28 340 КБ/сек (≈27.7 МБ/с)

Таблица 3: Результаты токенизации

3.5 Вывод

Реализованный токенизатор обеспечивает производительность ≈28 МБ/с и корректно справляется с основной массой текста. Для повышения качества необходимо доработать обработку специальных случаев (технические термины, URL, кириллический регистр).

4 Закон Ципфа

4.1 Цель работы

Построить частотный словарь терминов корпуса и проверить выполнение закона Ципфа:

$$f(r) \approx \frac{C}{r}$$

где $f(r)$ — частота терма с рангом r, C — константа.

4.2 Реализация

На основе токенизатора реализована программа `freq_counter.cpp`, которая:

- Считывает документы из SQLite (до 10 000 для ускорения).
- Токенизирует текст по тем же правилам.
- Подсчитывает частоту каждого уникального терма через `std::map`.
- Сортирует термы по убыванию частоты.
- Сохраняет результат в `freq_data.csv` (ранг, частота, терм).

График строится скриптом `zipf_plot.py` (matplotlib) в лог-лог шкале: эмпирическое распределение и теоретическая кривая C/r.

4.3 Результаты

На графике в логарифмических координатах эмпирическая кривая близка к прямой, что подтверждает выполнение закона Ципфа. Отклонения наблюдаются:

- В области малых рангов — самые частотные слова (предлоги, союзы) убывают не идеально по $1/r$.
- В «хвосте» распределения — ступенчатость из-за большого числа редких терминов с частотой 1–2.

Среди самых частотных токенов обнаружены HTML-сущности и служебные слова, связанные со структурой страниц Habr (навигация, теги). Это указывает на необходимость более агрессивной очистки HTML.

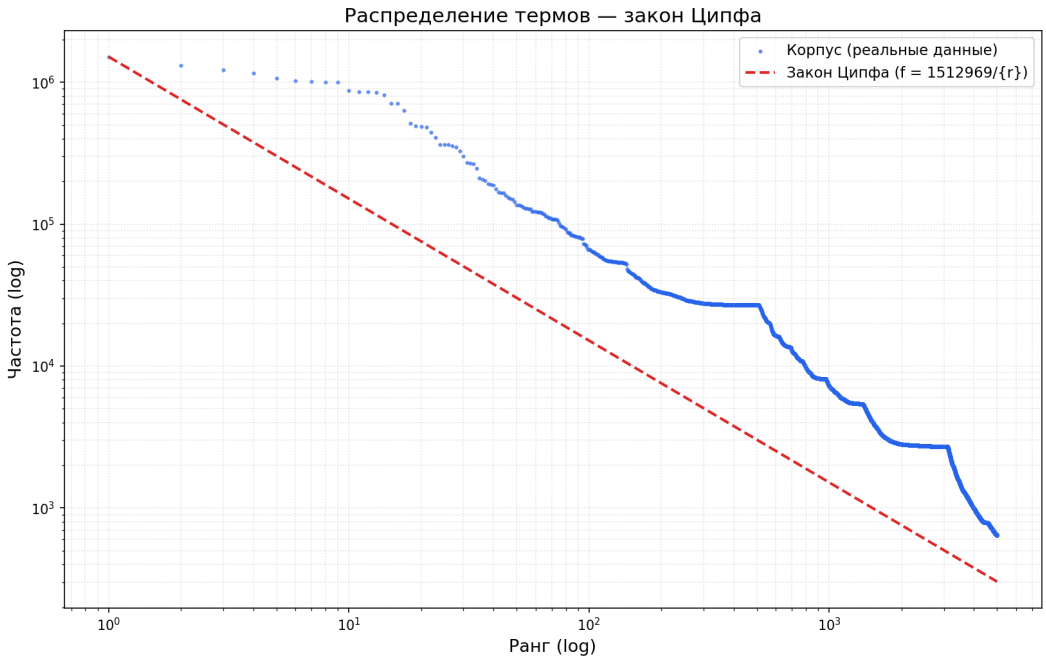

Рис. 1: Закон Ципфа: распределение «ранг–частота» в лог-лог шкале (эмпирика и модель C/r)

4.4 Вывод

Корпус подчиняется закону Ципфа, что характерно для текстов на естественном языке. Наличие «мусорных» высокочастотных токенов (разметка, код) подтверждает важность фильтрации стоп-слов и очистки HTML перед индексацией.

5 Стемминг

5.1 Цель работы

Реализовать компонент морфологической нормализации для приведения словоформ к единой основе. Стемминг должен применяться как при индексации, так и при обработке поисковых запросов.

5.2 Метод решения

Использование готовых NLP-библиотек запрещено, поэтому реализован собственный суффиксный стеммер (Suffix Stripping). Стеммер встроен в `indexer.cpp` и `searcher.cpp`.

Алгоритм:

1. Слова короче 5 байт (UTF-8) не обрабатываются — это предлоги, союзы и аббревиатуры.
2. Проверяется набор суффиксов, отсортированных по убыванию длины (от длинных к коротким), чтобы более специфичные правила срабатывали раньше.
3. Если суффикс найден и оставшаяся основа не короче 4 символов — суффикс отсекается.
4. Применяется только одно отсечение (не итеративно).

Набор суффиксов:

- Английские: `-ization`, `-fulness`, `-iveness`, `-ement`, `-ation`, `-ness`, `-ment`, `-able`, `-ible`, `-ing`, `-ous`, `-ful`, `-ity`, `-ed`, `-ly`, `-er`, `-es`, `-al`, `-en`.
- Русские: `-ость`, `-ение`, `-ание`, `-ками`, `-ться`, `-ного`, `-ого`, `-ему`, `-ями`, `-ами`, `-ных`, `-ной`, `-ает`, `-ить`, `-ать`, `-ая`, `-ый`, `-ий`, `-ое`, `-ые` и др.

5.3 Результаты тестирования

Автотесты (`stemmer_test.cpp`) подтвердили корректность работы на 20 тестовых случаях:

- `development` → `develop` (корректно)
- `programming` → `programm` (основа совпадает)
- `comfortable` → `comfort` (корректно)
- `darkness` → `dark` (корректно)
- `разработки` → `разработк` (корректно)
- `телефонами` → `телефон` (корректно)
- `программирование` → `программиров` (корректно)
- `app` → `app` (короткое слово — без изменений)

5.4 Влияние на качество поиска

Улучшение (рост Recall): По запросу «разработка» теперь находятся документы со словами «разработки», «разработку», «разработке». Без стемминга мы теряли бы значительную часть релевантных документов.

Ухудшение (падение Precision):

- Омонимия основ: «банк» (финансы) и «банка» (ёмкость) могут склеиться.
- Чрезмерное обрезание: «данные» и «дан» могут сойтись к одной основе.

Возможные улучшения: словари исключений для проблемных слов; гибридный поиск — сначала по точному совпадению (с бонусом), затем по стемам.

5.5 Вывод

Реализованный суффиксный стеммер корректно обрабатывает основные словоформы русского и английского языков. Все 20 автотестов проходят успешно. Стемминг существенно повышает полноту поиска, хотя и может вносить шум за счёт ложных слияний.

6 Булев индекс

6.1 Цель работы

Построить обратный (инвертированный) индекс для выполнения булевых запросов по корпусу документов.

6.2 Формат данных

Индекс разделён на два бинарных файла:

- `documents.bin` **(прямой индекс):** Для каждого документа записаны ID, URL и заголовок.
  - Структура: [`DocID`, 4 байта] [`UrlLen`, 8 байт] [`URL`] [`TitleLen`, 8 байт] [`Title`].
- `inverted.bin` **(обратный индекс):** Для каждого терма записан список документов.
  - Структура: [`TermLen`, 4 байта] [`Term`] [`DocCount`, 4 байта] [$\texttt{DocID}_1$] ... [$\texttt{DocID}_N$].

6.3 Реализация

Индексатор (`indexer.cpp`) последовательно читает документы из SQLite и строит индекс в оперативной памяти.

Структуры данных:

- Хеш-таблица с цепочками коллизий, 65 537 слотов.
- Хеш-функция FNV-1a.
- Каждый элемент `TermEntry` содержит слово, счётчик документов и связанный список `PostingNode` (doc_id).

Для каждого терма выполняется стемминг (та же функция `apply_stemmer`), после чего doc_id добавляется в posting list. Дубликаты не допускаются — проверка по последнему элементу списка.

Заголовок документа извлекается из HTML-тега `<title>`; если тег не найден, используется URL.

6.4 Результаты

- Документов проиндексировано: 30 000.
- Уникальных термов в индексе: 1 243 817.
- Размер `inverted.bin`: 547 МБ.
- Размер `documents.bin`: 4.8 МБ.

6.5 Масштабируемость

Текущая реализация (In-Memory) подходит для корпусов до ∼100 000 документов. Для корпусов в миллионы документов потребуется переход на алгоритм SPIMI (Single-Pass In-Memory Indexing): накопление индекса порциями с дампом на диск и финальным слиянием (K-way Merge).

6.6 Вывод

Обратный индекс построен и сериализован в компактный бинарный формат. Posting lists автоматически отсортированы (документы обрабатываются последовательно), что обеспечивает выполнение булевых операций за линейное время.

7 Булев поиск и веб-интерфейс

7.1 Цель работы

Реализовать поисковый движок, выполняющий булевы запросы по построенному индексу, с двумя интерфейсами: командная строка (CLI) и веб (HTML-форма).

7.2 Метод решения

7.2.1 Backend (C++): searcher

Программа `searcher.cpp` загружает `documents.bin` и `inverted.bin` в RAM при старте. Для каждого слова запроса применяется тот же стеммер, что и при индексации.

Булевы операции над отсортированными posting lists:

- `&&` (AND): `std::set_intersection` — пересечение.
- `||` (OR): `std::set_union` — объединение.
- `!` (NOT): `std::set_difference` — разность с множеством всех документов.

Слова без оператора между ними трактуются как AND.

7.2.2 Frontend (Python): web_search.py

Веб-сервер на базе `http.server` (порт 8080). Принимает запрос из формы, запускает `./searcher` через `subprocess`, парсит вывод (формат: `заголовок<TAB>url`) и генерирует HTML-страницу с результатами.

7.3 Синтаксис запросов

Оператор	Значение	Пример
`&&`	И (пересечение)	`android && kotlin`
`||`	ИЛИ (объединение)	`flutter || swift`
`!`	НЕ (исключение)	`android && !java`
пробел	И (по умолчанию)	`ios swift`

Таблица 4: Синтаксис булевых запросов

7.4 Тестирование

1. **Проверка AND**: запросы `android` и `kotlin` по отдельности, затем `android && kotlin`. Результат AND является подмножеством каждого из одиночных — корректно.
2. **Проверка OR**: результат `flutter || swift` содержит объединение документов из обоих одиночных запросов — корректно.
3. **Проверка NOT**: запрос `android && !java`. Выборочная проверка показала, что в найденных документах слово «java» отсутствует.
4. **Автотест стеммера**: программа `stemmer_test` проверяет 20 тестовых случаев и завершается с кодом 0 при успехе.

7.5 Производительность

Поиск выполняется мгновенно (<1 мс) для любых запросов, так как весь индекс находится в RAM. Задержка может возникнуть только при запросах со стоп-словами и оператором OR, когда результирующий список содержит почти все документы базы.

7.6 Вывод

Реализована полнофункциональная система булева поиска с CLI- и веб-интерфейсом. Использование бинарного индекса и операций над отсортированными списками обеспечивает высокую производительность. Для дальнейшего улучшения можно добавить поддержку скобок (приоритет операторов), ранжирование результатов (TF-IDF) и построение сниппетов.